Web Application Attack Simulation and Layered Security Hardening

WAF Rate Limiting and Defensive Controls

Project overview

We created a vulnerable web application to understand how common attacks affect a website and the people who use it. We then built a separate hardened version and added security controls to address the problems we found. By repeating the same attacks, we could see how the changes protected information and improved the way the website handled suspicious requests.

The project follows a simple process: find a weakness, observe what happens, apply a fix and try the same request again. It shows how application changes, a Web Application Firewall and traffic controls work together to improve security.

Team members

Team member	Roll number
Sidharth Kamath	16010124126
Soham Kanase	16010124128
Atharva Kamat	16010124125
Arnav Vaidya	

About the website

LegacyDesk is an employee service portal. Employees can sign in, raise support tickets, follow conversations, update their contact details and download useful resources. An administrator can access the administration page to view basic portal information.

The website was designed to look and behave like a normal service portal. Its original version contained weaknesses in login, ticket access, comments, account changes, downloads and request handling. These features gave us practical ways to observe how an application can fail when it trusts requests without enough checking.

We kept the original website available and created a separate hardened version. This allowed us to compare the same features before and after adding protection.

Objectives

- Create a vulnerable website that demonstrates common web security problems.
- Understand how attacks can expose private information, change account details or cause unnecessary server work.
- Fix the weaknesses in the application and add protective layers around it.
- Repeat the same attacks and confirm that ordinary employee features still work.

System architecture and defensive layers



Each layer checks the request before it reaches the next part of the website.

Requests pass through the protective layers before reaching application data.

The hardened website receives requests through an Nginx reverse proxy. The proxy handles the secure connection and controls repeated traffic. The WAF checks requests for common attack patterns before passing them to the application. The application then checks the user, validates the input and decides whether the requested action is allowed.

Layer	What we added
Secure connection	HTTPS protects the connection between the browser and the website.
Reverse proxy	Request and connection controls reduce unnecessary traffic reaching the application.
Web Application Firewall	Suspicious requests are checked and common attack patterns are blocked.
Application	Safer database queries, output encoding, permission checks and stronger sessions protect website features.
Request handling	User-based controls and report caching reduce repeated work.
Logging	Security events help us understand which requests were accepted or rejected.

The layers support each other. We fixed the application itself and also added controls at the proxy, WAF and browser level.

What we found

SQL injection

We entered a database condition into the sign-in form instead of a normal username. The website treated it as part of its database query and let us enter an administrator account without the correct password. This showed that the login form was trusting user input too much.

V01 SQL injection

http://127.0.0.1:8080 **HTTP 303**

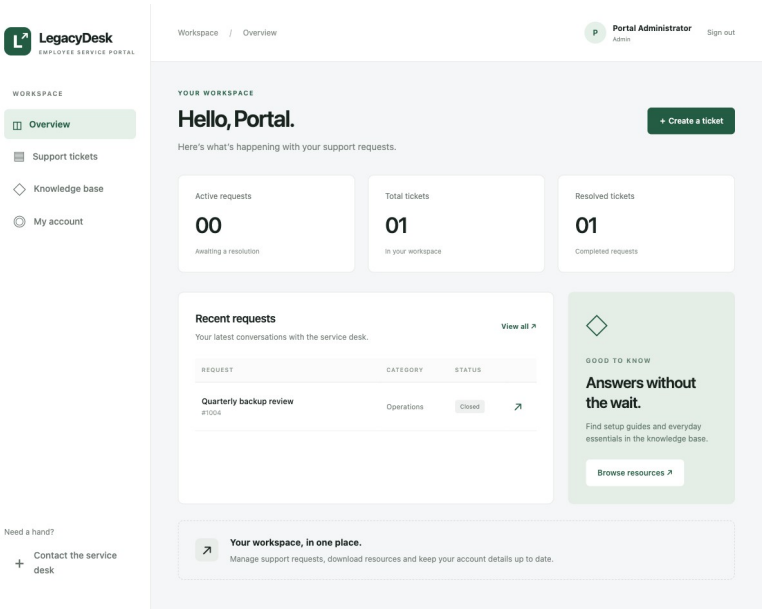
Request

```
POST /login
{
  "Content-Type": "application/x-www-form-urlencoded"
}
username=%27+OR+1%3D1+--+&password=wrong
```

Observed response

```
{
  "Content-Type": "text/html; charset=utf-8",
  "Location": "/",
  "Set-Cookie": "user_id=1; Path=/"
}
```

The sign-in request selected the administrator account.



The website opened the administrator workspace.

Stored cross site scripting

We posted a comment containing browser code and opened the ticket again. The website ran the code as part of the page. The comment changed its own content, showing that a saved message could affect what another user sees.

V02 Stored cross site scripting

http://127.0.0.1:8080 **HTTP 200**

Request

```
GET /ticket?id=1001
{
  "Cookie": "user_id=2"
}
```

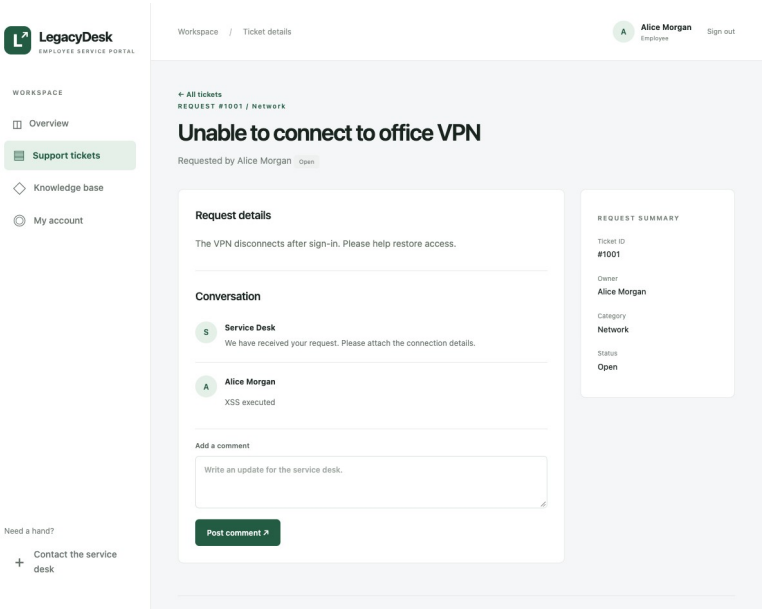
Observed response

```
{
  "Content-Type": "text/html; charset=utf-8"
}

Stored response markup:
<svg onload="this.outerHTML='<p>XSS executed</p>'"></svg>

Browser execution is captured separately in V02-before-browser.png.
```

The saved comment was returned as page code.



The browser ran the comment code inside the conversation.

Access to other employee tickets

We signed in as an employee and changed the ticket identifier in a request. The website opened a ticket belonging to someone else and displayed private payroll information. It checked that we were signed in, but did not check whether we were allowed to see that ticket.

V03 Broken object access control

http://127.0.0.1:8080 **HTTP 200**

Request

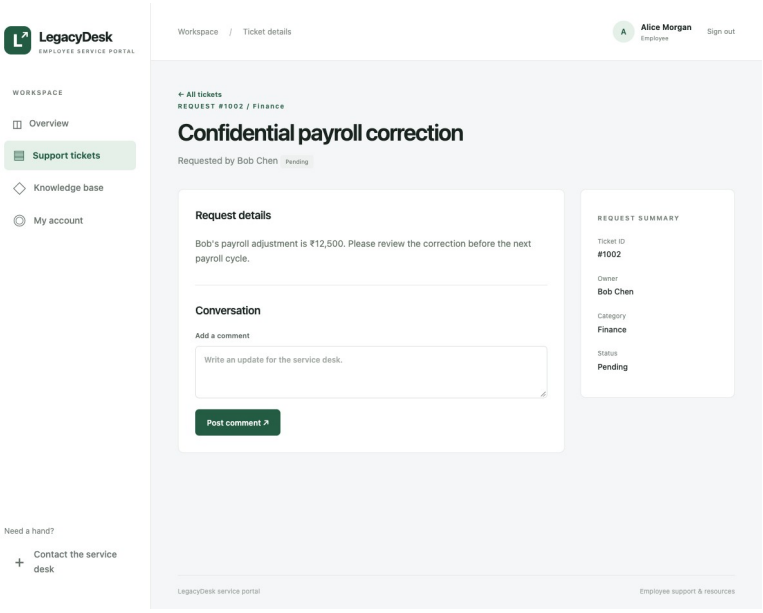
```
GET /ticket?id=1002
{
  "Cookie": "user_id=2"
}
```

Observed response

```
{
  "Content-Type": "text/html; charset=utf-8"
}

 ← All tickets REQUEST #1002 / Finance Confidential payroll correction Requested by Bob Chen Pending
Request details Bob's payroll adjustment is ₹12,500. Please review the correction before the next
payroll cycle. Conversation Add a comment Post comment ↗ REQUEST SUMMARY Ticket ID #1002 Owner Bob
Chen Category Finance Status Pending
```

Changing the requested ticket exposed private employee information.



The signed-in employee could read a ticket owned by someone else.

Weak authentication and sessions

We changed the identity value stored in the browser cookie. The website trusted the changed value and gave us administrator access. We also found that the original account passwords were stored in a readable form. A person who could copy those records would not need to guess the passwords.

V04 Forged identity cookie

http://127.0.0.1:8080 **HTTP 200**

Request

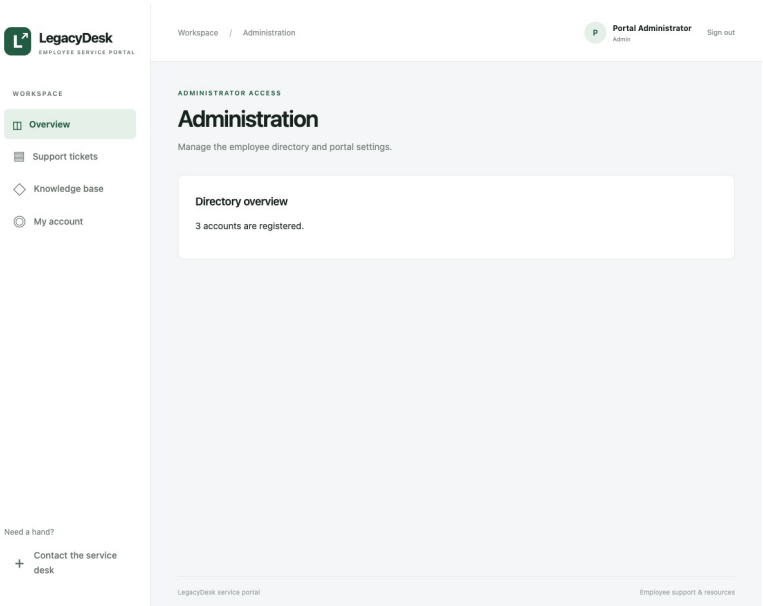
```
GET /admin
{
  "Cookie": "user_id=1"
}
```

Observed response

```
{
  "Content-Type": "text/html; charset=utf-8"
}

Administrator page returned to the forged cookie.
 Administration · LegacyDesk L ↗ LegacyDesk EMPLOYEE SERVICE PORTAL WORKSPACE ◫ Overview ▤ Support
tickets ◇ Knowledge base ◎ My account Need a hand? + Contact the service desk Workspace /
Administration P Portal Administrator Admin Sign out ADMINISTRATOR ACCESS Administration Manage the
employee directory and portal settings. Directory overview 3 accounts are registered. LegacyDesk
service portal Employee support & resources
```

The changed cookie was accepted as an administrator identity.



The administrator page opened after the identity cookie was changed.

Cross site request forgery

While an employee was signed in, we submitted a contact form from a separate page. It changed the employee email address without using the account form inside the portal. The website accepted the browser session without checking where the change came from.

V05 Cross site request forgery

http://127.0.0.1:8080 **HTTP 303**

Request

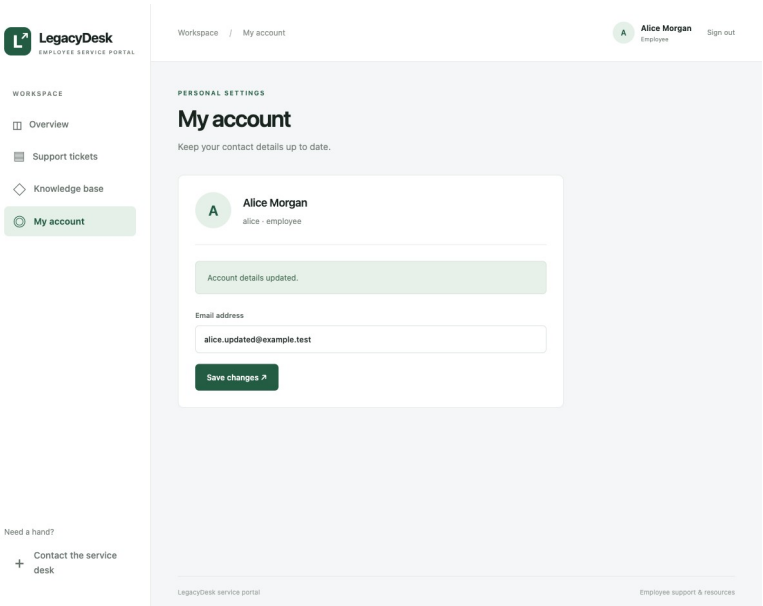
```
POST /account/email
{
  "Content-Type": "application/x-www-form-urlencoded",
  "Cookie": "user_id=2",
  "Origin": "http://127.0.0.1:8091"
}
email=alice.updated%40example.test
```

Observed response

```
{
  "Content-Type": "text/html; charset=utf-8",
  "Location": "/account?saved=1"
}


Email after request: alice.updated@example.test (changed without token)
```

The outside request changed the account email.



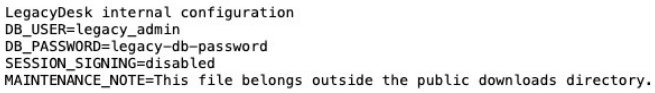
The account showed the changed email after the request.

Path traversal in file downloads

We changed the filename in a download request so that it pointed outside the public downloads folder. The website returned an internal configuration file instead of the employee handbook. This showed that the download feature could expose files that were meant to stay private.



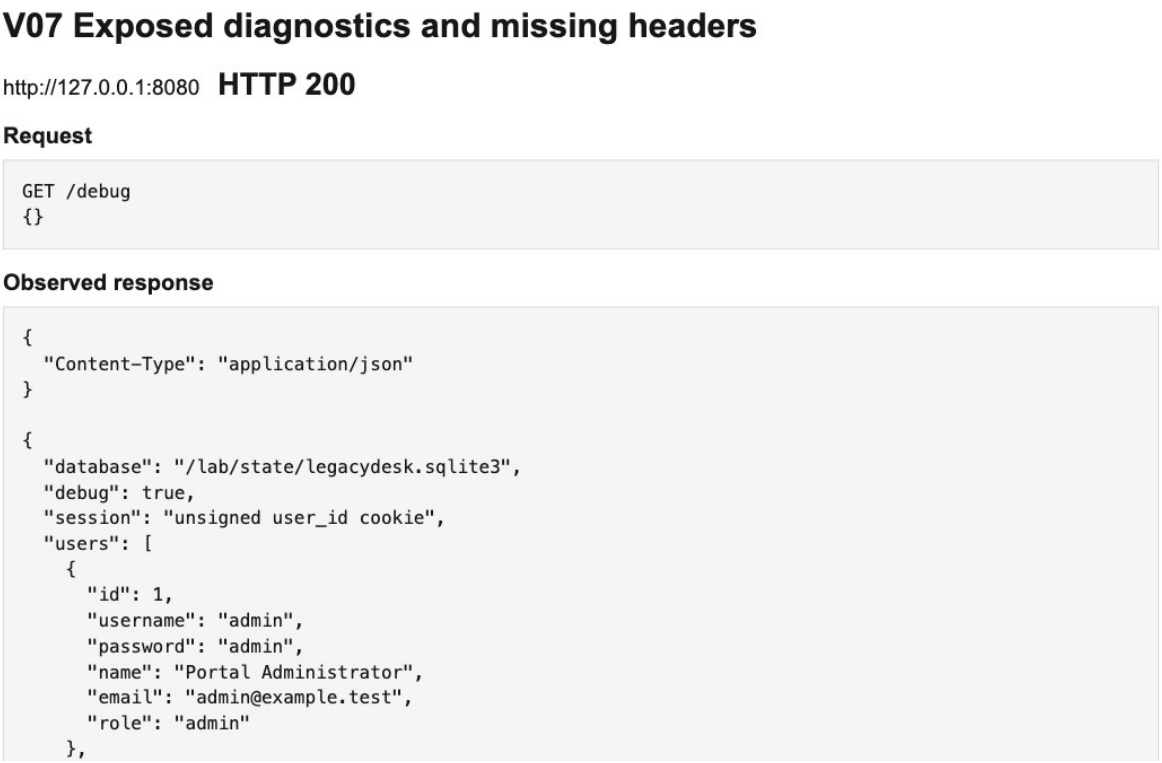
The download request reached a private configuration file.



The internal file contents appeared in the browser.

Exposed internal information

We opened a diagnostic page without signing in. It revealed account details, readable passwords and information about how the application worked. We also found that the website was missing browser security headers and showed more error information than a visitor needed.



V07 Exposed diagnostics and missing headers

http://127.0.0.1:8080 **HTTP 200**

Request

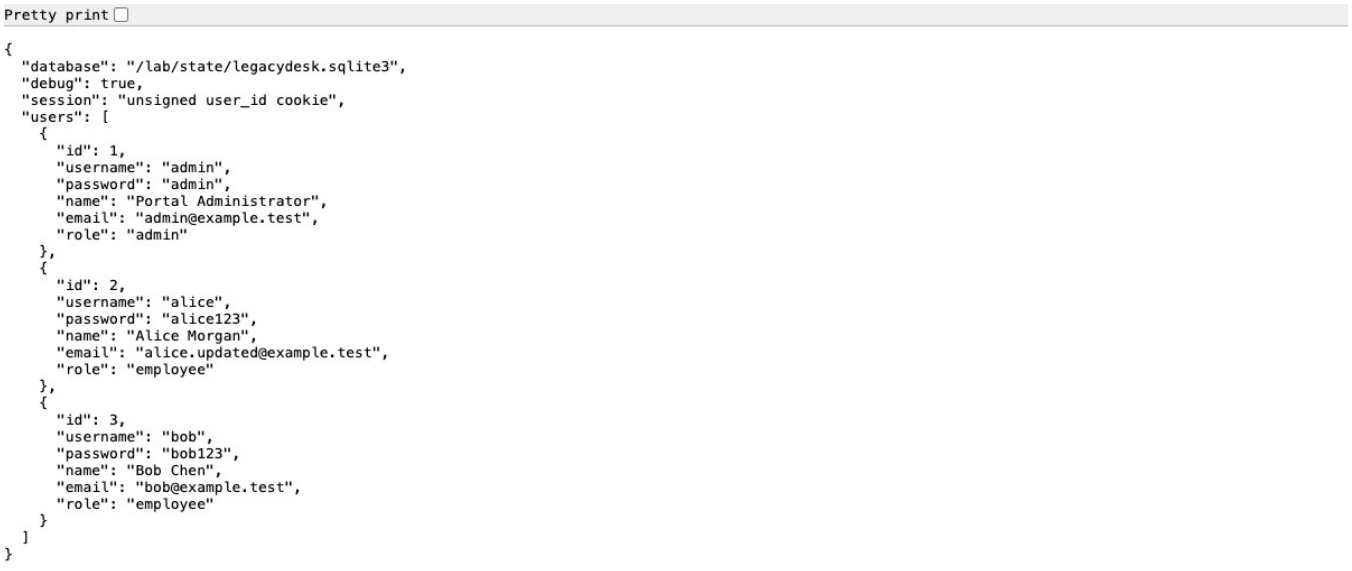
```
GET /debug
{}
```

Observed response

```
{
  "Content-Type": "application/json"
}

{
  "database": "/lab/state/legacydesk.sqlite3",
  "debug": true,
  "session": "unsigned user_id cookie",
  "users": [
    {
      "id": 1,
      "username": "admin",
      "password": "admin",
      "name": "Portal Administrator",
      "email": "admin@example.test",
      "role": "admin"
    },
```

The public diagnostic page exposed account information.



Pretty print ☐


```
{
  "database": "/lab/state/legacydesk.sqlite3",
  "debug": true,
  "session": "unsigned user_id cookie",
  "users": [
    {
      "id": 1,
      "username": "admin",
      "password": "admin",
      "name": "Portal Administrator",
      "email": "admin@example.test",
      "role": "admin"
    },
    {
      "id": 2,
      "username": "alice",
      "password": "alice123",
      "name": "Alice Morgan",
      "email": "alice.updated@example.test",
      "role": "employee"
    },
    {
      "id": 3,
      "username": "bob",
      "password": "bob123",
      "name": "Bob Chen",
      "email": "bob@example.test",
      "role": "employee"
    }
  ]
}
```


Internal account records were visible without signing in.

Unrestricted resource consumption

This vulnerability is unrestricted resource consumption, which can cause denial of service (DoS). Repeated report requests consume capacity needed by normal visitors. Our earlier bounded check showed V1 accepting all 40 requests. In a separate local demonstration, we used two workers and four-second report jobs: V1 returned HTTP 503 during traffic and recovered after the script stopped. The traffic came from one source, so this was a local DoS demonstration rather than a distributed denial-of-service (DDoS) test.



V08 Unrestricted expensive requests

http://127.0.0.1:8080 **40 requests / 4 workers**

Request

```
GET /reports/export
Authenticated Alice session
Bounded local request batch
```

Observed response


```
{
  "requests": 40,
  "concurrency": 4,
  "elapsed_seconds": 2.055,
  "throughput_rps": 19.46,
  "status_counts": {
    "200": 40
  },
  "median_latency_ms": 204.69,
  "p95_latency_ms": 207.64,
  "limitation": "Bounded local probe demonstrates missing admission controls; does not establish DoS,
DDoS resistance or system resource consumption."
}
```


Earlier bounded check: V1 accepted all 40 report requests.

How we fixed the vulnerabilities

SQL injection

We changed the login so that the database treats a username as text rather than an instruction. The password is checked separately, and unexpected input is rejected. We also added a Web Application Firewall to filter suspicious requests. When we repeated the attack, it no longer gave us administrator access.



V01 SQL injection

https://127.0.0.1:8443 **HTTP 403**

Request

```
POST /login
Identity: Alice session [redacted]
Origin: https://127.0.0.1:8443
{
  "username": "' OR 1=1 -- ",
  "password": "[redacted]",
  "csrf_token": "[redacted]"
}
```

Observed response

```
{
  "Content-Type": "text/html",
  "Strict-Transport-Security": "max-age=86400",
  "Content-Security-Policy": "default-src 'none'; style-src 'self'; img-src 'self'; form-action
'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'",
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "Referrer-Policy": "same-origin",
  "X-Request-ID": "e7ed16e6f83df980cd721ffc338b3059"
}

 403 Forbidden 403 Forbidden nginx
```

The repeated request was handled by the hardened website.

Fixing stored cross site scripting

We changed how comments are displayed so that user content appears as text and cannot become part of the page code. We also added browser security rules and WAF filtering. The repeated request was blocked, and a separate application check showed that the same comment stayed as harmless text.

V02 Stored cross site scripting

https://127.0.0.1:8443 **HTTP 403**

Request

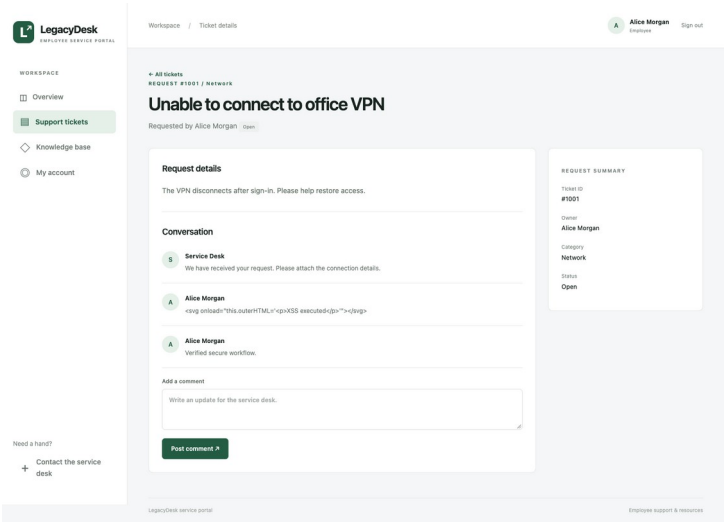
```
POST /comment
Identity: Alice session [redacted]
Origin: https://127.0.0.1:8443
{
  "ticket_id": "1001",
  "body": "<svg onload=\"this.outerHTML='<p>XSS executed</p>'\"></svg>",
  "csrf_token": "[redacted]"
}
```

Observed response

```
{
  "Content-Type": "text/html",
  "Strict-Transport-Security": "max-age=86400",
  "Content-Security-Policy": "default-src 'none'; style-src 'self'; img-src 'self'; form-action
'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'",
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "Referrer-Policy": "same-origin",
  "X-Request-ID": "2b07351947a4827f9cf8cf3ed49cc153"
}

 403 Forbidden 403 Forbidden nginx
```

The repeated request was handled by the hardened website.



The application displays the comment as text.

Fixing access to other employee tickets

We added an ownership check whenever a ticket is opened or a comment is posted. Employees can work with their own tickets, while administrators keep their intended access. When we repeated the request using the same employee account, the private ticket was denied and the employee could still open their own ticket.

V03 Broken object access control

https://127.0.0.1:8443 **HTTP 403**

Request

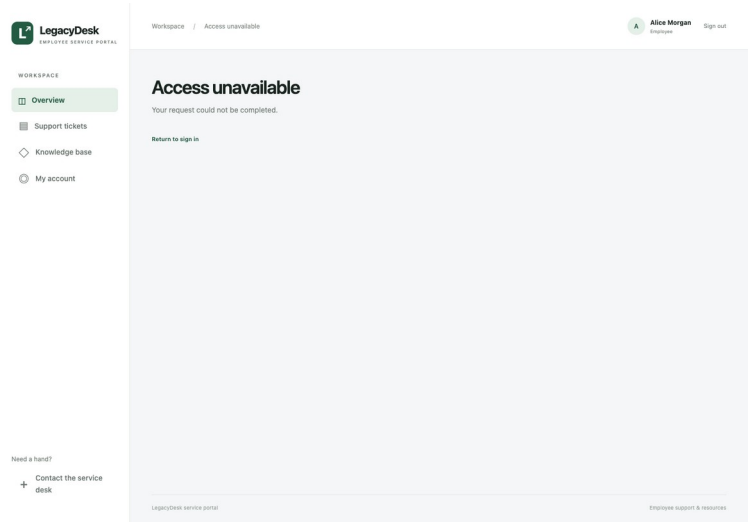
```
GET /ticket?id=1002
Identity: Alice session [redacted]
Origin: None
{}
```

Observed response

```
{
  "Content-Type": "text/html; charset=utf-8",
  "Strict-Transport-Security": "max-age=86400",
  "Content-Security-Policy": "default-src 'none'; style-src 'self'; img-src 'self'; form-action
'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'",
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "Referrer-Policy": "same-origin",
  "X-Request-ID": "1284dad58fd570c716d241a7b132ef6b"
}

 Access unavailable · LegacyDesk L ↗ LegacyDesk EMPLOYEE SERVICE PORTAL WORKSPACE ◫ Overview ▤ Support
tickets ◇ Knowledge base ◎ My account Need a hand? + Contact the service desk Workspace / Access
unavailable A Alice Morgan Employee Sign out Access unavailable Your request could not be completed.
Return to sign in LegacyDesk service portal Employee support & resources
```

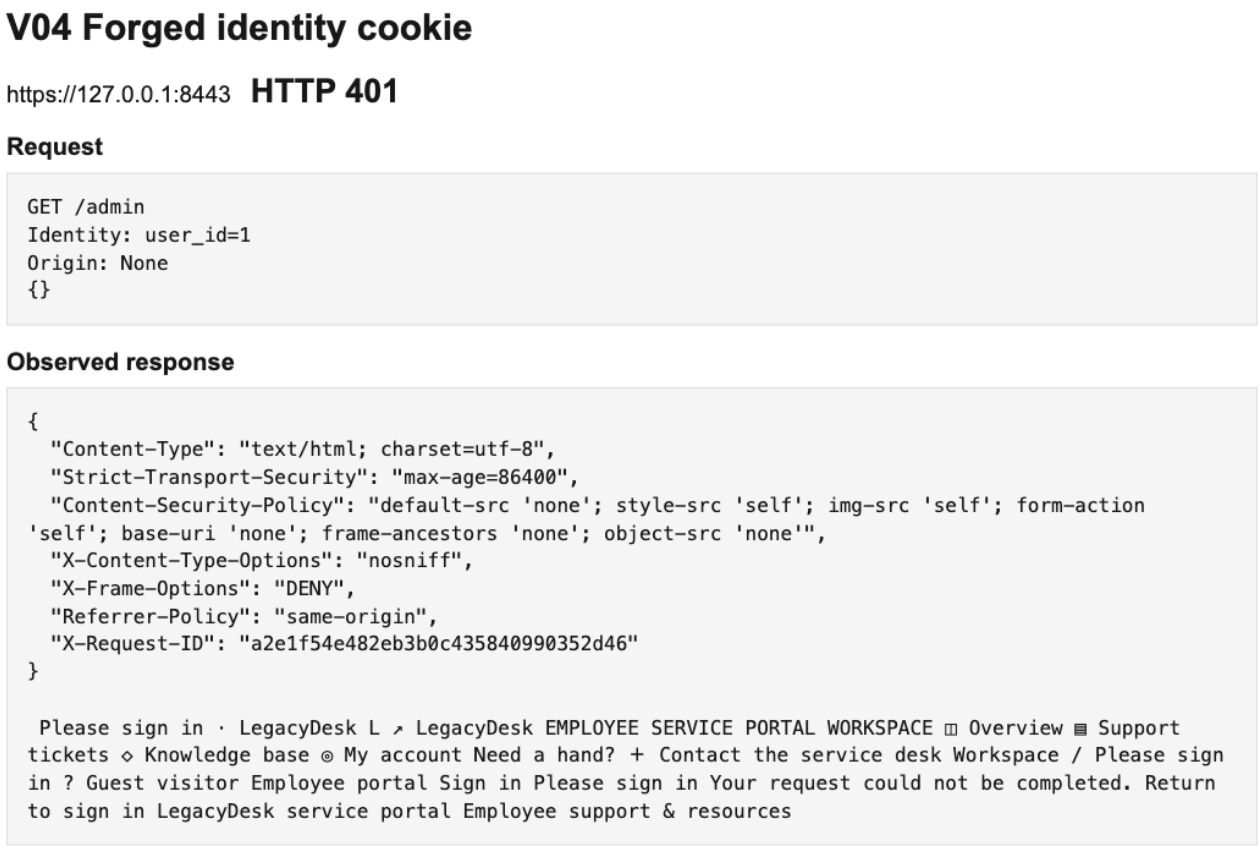
The repeated request was handled by the hardened website.



The hardened website denies access to another employee ticket.

Fixing weak authentication and sessions

We replaced the trusted identity cookie with a random session that the server checks. Sessions are replaced at sign-in, expire when no longer valid, and stop working after sign-out. Passwords are now stored as hashes instead of readable text. Changing the old cookie no longer creates an administrator session, and an employee account still cannot open administration.



V04 Forged identity cookie

https://127.0.0.1:8443 **HTTP 401**

Request

```
GET /admin
Identity: user_id=1
Origin: None
{}
```

Observed response


```
{
  "Content-Type": "text/html; charset=utf-8",
  "Strict-Transport-Security": "max-age=86400",
  "Content-Security-Policy": "default-src 'none'; style-src 'self'; img-src 'self'; form-action
'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'",
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "Referrer-Policy": "same-origin",
  "X-Request-ID": "a2e1f54e482eb3b0c435840990352d46"
}
```


```
 Please sign in · LegacyDesk L ↗ LegacyDesk EMPLOYEE SERVICE PORTAL WORKSPACE ◫ Overview ▤ Support
tickets ◇ Knowledge base ◎ My account Need a hand? + Contact the service desk Workspace / Please sign
in ? Guest visitor Employee portal Sign in Please sign in Your request could not be completed. Return
to sign in LegacyDesk service portal Employee support & resources
```

The repeated request was handled by the hardened website.

Fixing cross site request forgery

We added a security token to forms that change account or website data. The server checks that the token belongs to the current session and that the request comes from the portal. We also strengthened cookie handling. Repeating the outside request was rejected, and the account email stayed unchanged.

V05 Cross site request forgery

https://127.0.0.1:8443 **HTTP 403**

Request

```
POST /account/email
Identity: Alice session [redacted]
Origin: http://127.0.0.1:8091
{
  "email": "alice.updated@example.test"
}
```

Observed response

```
{
  "Content-Type": "text/html; charset=utf-8",
  "Strict-Transport-Security": "max-age=86400",
  "Content-Security-Policy": "default-src 'none'; style-src 'self'; img-src 'self'; form-action
'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'",
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "Referrer-Policy": "same-origin",
  "X-Request-ID": "36c6bcaad5c02eb4f601ea175372e77c"
}

 Access unavailable · LegacyDesk L ↗ LegacyDesk EMPLOYEE SERVICE PORTAL WORKSPACE ◫ Overview ▤ Support
tickets ◇ Knowledge base ◎ My account Need a hand? + Contact the service desk Workspace / Access
unavailable A Alice Morgan Employee Sign out Access unavailable Your request could not be completed.
Return to sign in LegacyDesk service portal Employee support & resources
Email after request: unchanged (verified by a subsequent authenticated GET /account)
```

The repeated request was handled by the hardened website.

Fixing path traversal in file downloads

We changed the download feature to accept only approved filenames, each linked to a fixed file chosen by the application. Users must also be signed in before downloading. The WAF adds another check for suspicious file requests. The private-file request was denied, while the employee handbook still downloaded normally.



V06 Path traversal

https://127.0.0.1:8443 **HTTP 403**

Request

```
GET /download?file=..%2Fprivate%2Flegacy-config.txt
Identity: Alice session [redacted]
Origin: None
{}
```

Observed response

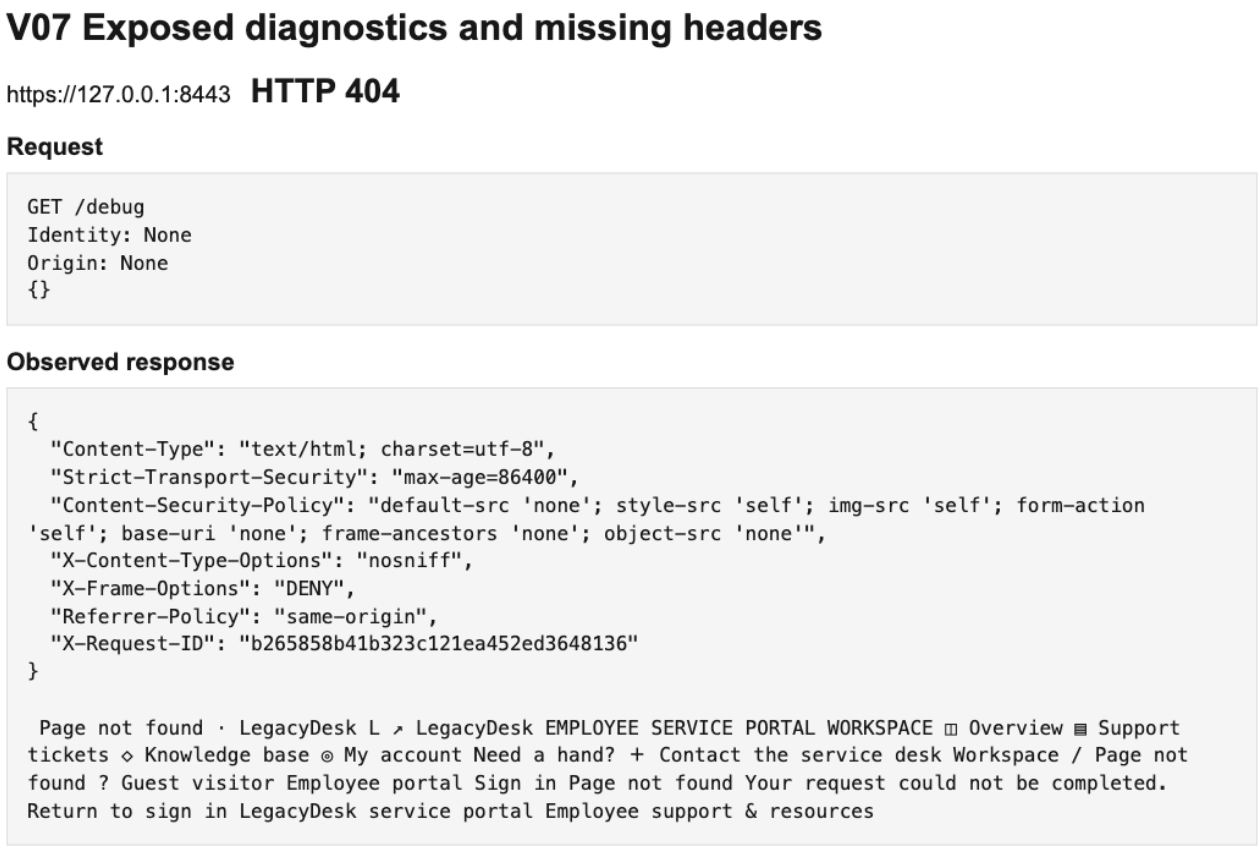
```
{
  "Content-Type": "text/html",
  "Strict-Transport-Security": "max-age=86400",
  "Content-Security-Policy": "default-src 'none'; style-src 'self'; img-src 'self'; form-action
'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'",
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "Referrer-Policy": "same-origin",
  "X-Request-ID": "cc59e2d11c0b5dd4066717a686387ca6"
}

 403 Forbidden 403 Forbidden nginx
```

The repeated request was handled by the hardened website.

Fixing exposed internal information

We removed the public diagnostic page and replaced detailed public errors with simple messages. We added secure response headers and HTTPS to protect how the browser loads and handles the website. The diagnostic request now returns a missing-page response without exposing account information.



V07 Exposed diagnostics and missing headers

https://127.0.0.1:8443 **HTTP 404**

Request

```
GET /debug
Identity: None
Origin: None
{}
```

Observed response

```
{
  "Content-Type": "text/html; charset=utf-8",
  "Strict-Transport-Security": "max-age=86400",
  "Content-Security-Policy": "default-src 'none'; style-src 'self'; img-src 'self'; form-action
'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'",
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "Referrer-Policy": "same-origin",
  "X-Request-ID": "b265858b41b323c121ea452ed3648136"
}

 Page not found · LegacyDesk L ↗ LegacyDesk EMPLOYEE SERVICE PORTAL WORKSPACE ◫ Overview ▤ Support
tickets ◇ Knowledge base ◎ My account Need a hand? + Contact the service desk Workspace / Page not
found ? Guest visitor Employee portal Sign in Page not found Your request could not be completed.
Return to sign in LegacyDesk service portal Employee support & resources
```

The repeated request was handled by the hardened website.

Fixing unrestricted resource consumption

We added request and connection limits at Nginx, a quota of six report requests per account per minute, and a 30-second report cache. Excess requests receive HTTP 429 instead of triggering more report work. In the separate local capacity demonstration, V2 rejected repeated report requests while normal ticket pages continued to return HTTP 200. These controls preserve availability for the tested workload; they do not identify genuine users automatically or prove resistance to every DoS attack.



V08 Unrestricted expensive requests

https://127.0.0.1:8443 **40 requests / 4 workers**

Request

```
GET /reports/export
Authenticated Alice session
Bounded local request batch
```

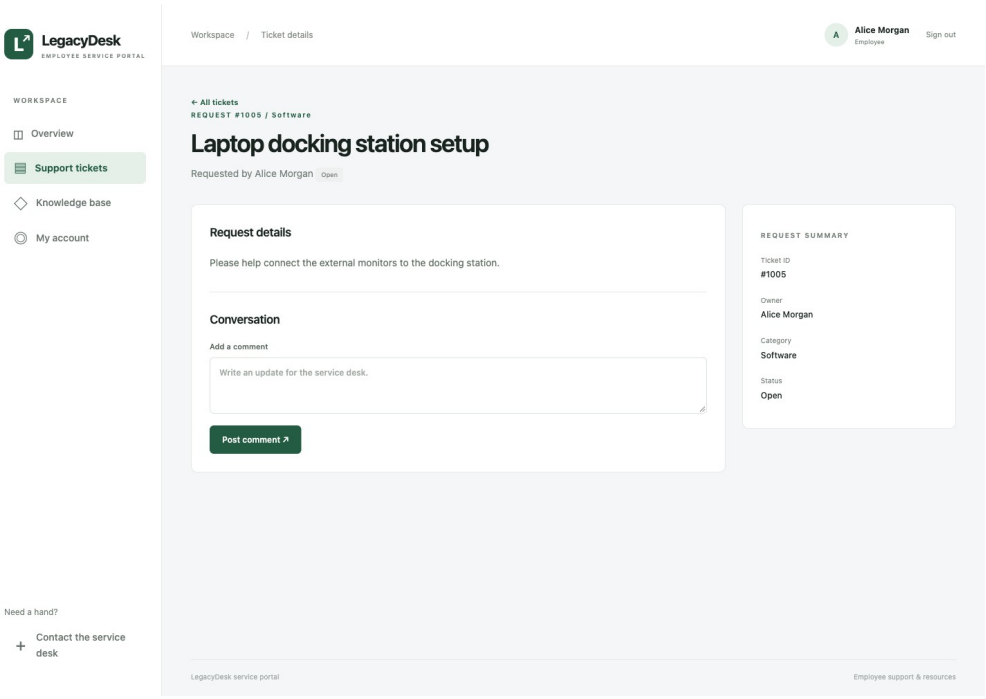
Observed response


```
{
  "requests": 40,
  "workers": 4,
  "elapsed_seconds": 0.108,
  "status_counts": {
    "200": 5,
    "429": 35
  },
  "completed_responses_rps": 369.64,
  "median_latency_ms": 5.46,
  "p95_latency_ms": 41.59,
  "limitation": "Same request count and concurrency as v1; includes TLS and rejected requests.
Throughput is not accepted report throughput or proof of DDoS resistance."
}
```

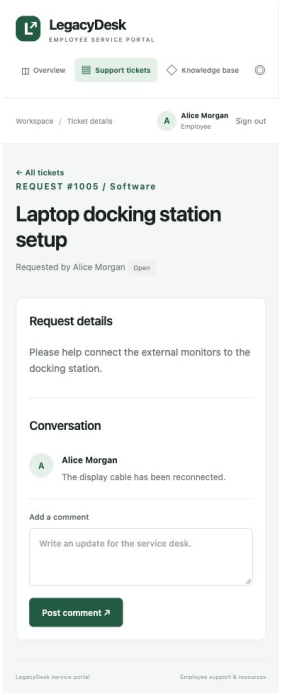

Earlier bounded check: V2 accepted 5 reports and limited 35 with HTTP 429.

Normal website use after the changes

After adding the security controls, we checked the features an employee would use every day. Signing in, creating a ticket, adding a normal comment and downloading the handbook continued to work. The website also remained usable on a smaller screen.



An employee creates a support ticket on the hardened website.

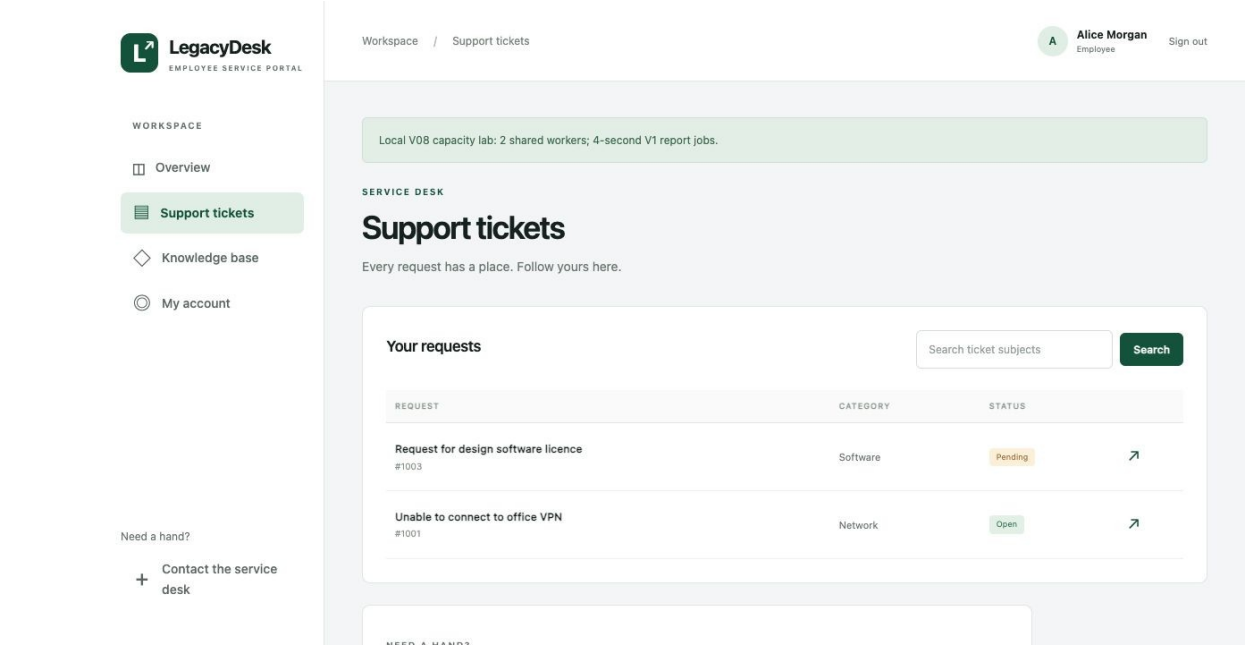


The ticket remains usable on a smaller screen.

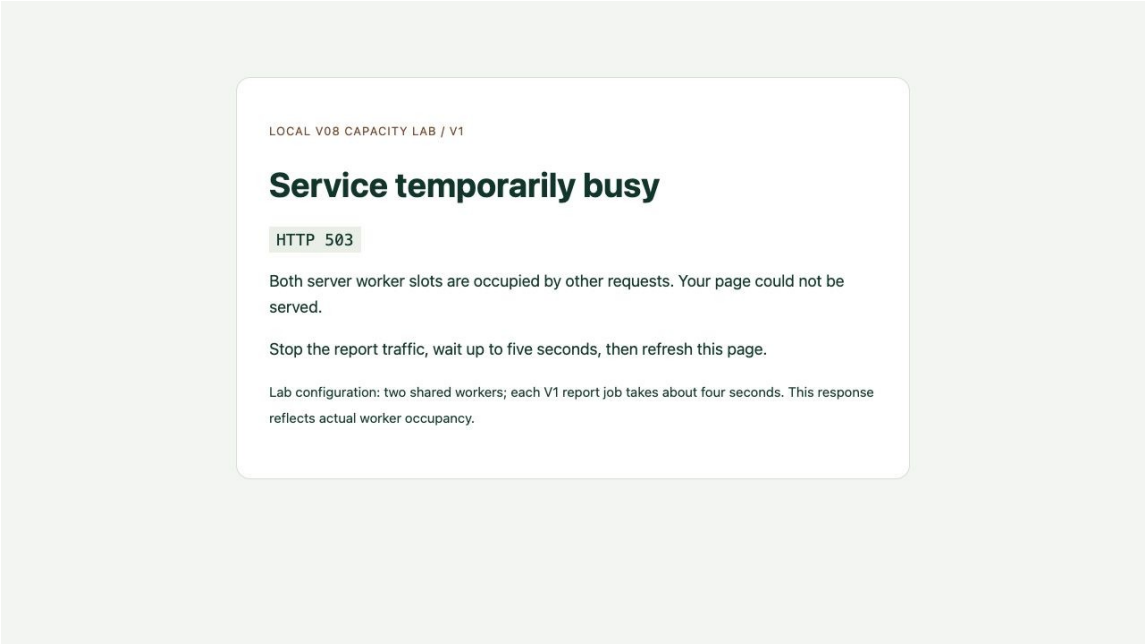
The comparison showed that the same service portal could keep its normal features while rejecting the harmful requests we had demonstrated.

V08 local availability demonstration

These browser captures show the separate local capacity lab on 18 October 2026. It uses two shared workers and four-second V1 report jobs to make request contention visible.



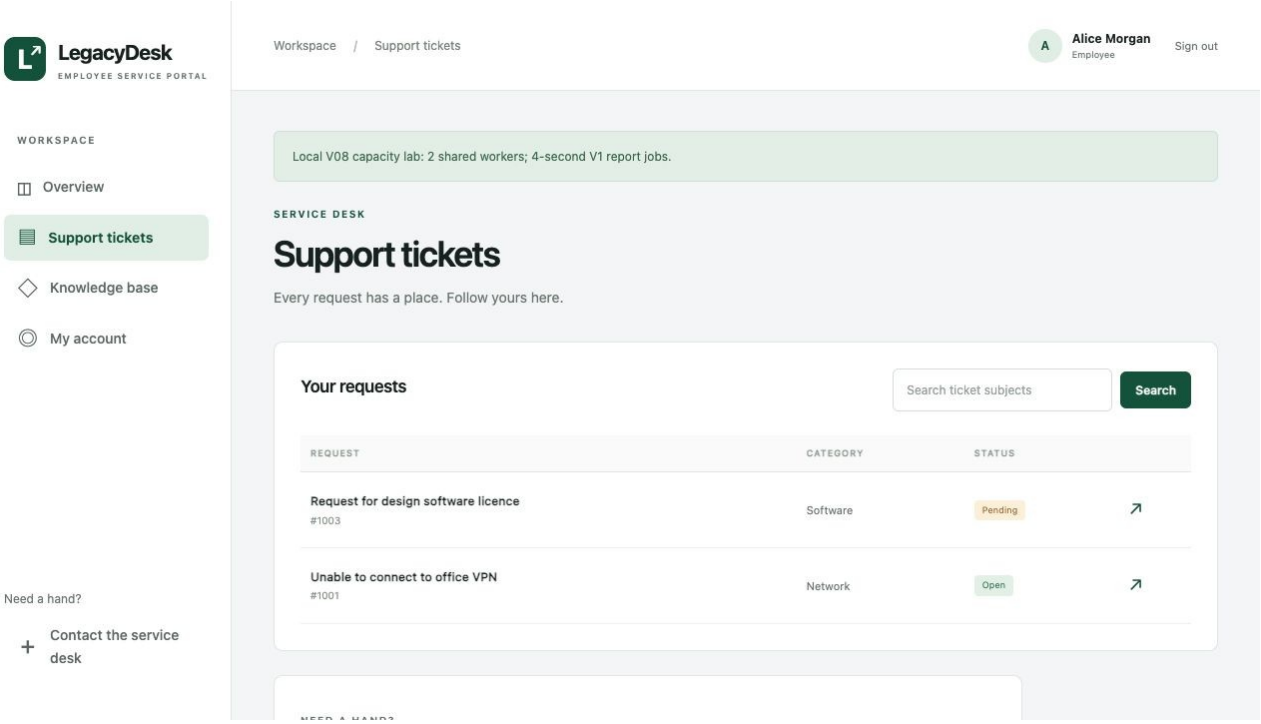
V1 before traffic: the ticket page loads normally in the local capacity lab.



V1 during report traffic: both workers are occupied and the browser receives HTTP 503.

V08 recovery after stopping traffic

After stopping the traffic script with Ctrl+C and allowing running jobs to finish, the same browser ticket page became available again without restarting the server.



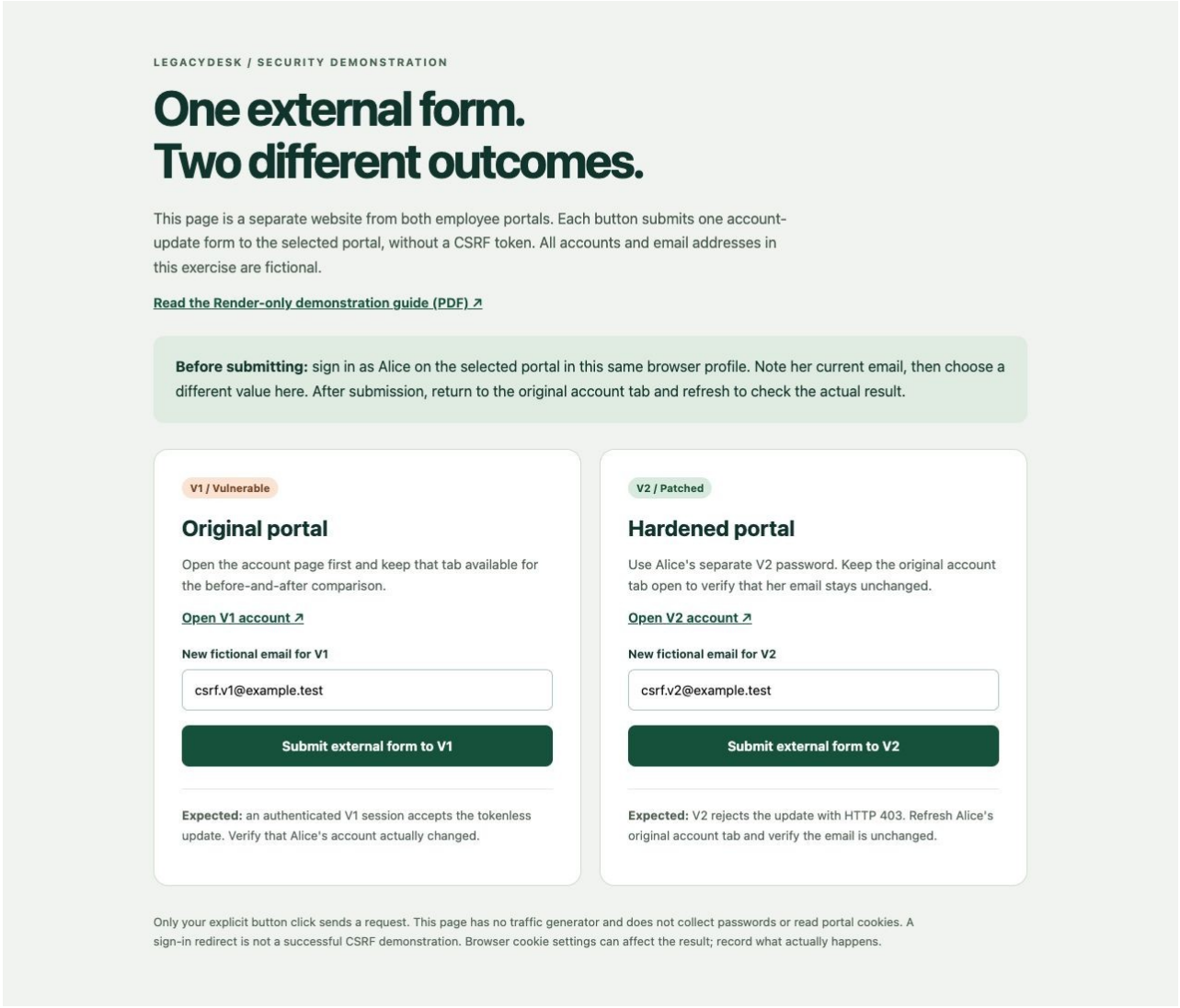
V1 after traffic stops: the ticket page is available again in the same browser tab.

V2 was checked separately over HTTPS with the local CA certificate. During the short verification, excess report requests returned HTTP 429 while the tested normal pages returned HTTP 200. Its report quota and cache reduced repeated work.

The screenshot above records V1 recovery. V2 availability is supported by the recorded HTTPS checks; no V2 browser capture from this capacity test is presented here. This deliberately constrained local test does not establish resistance to distributed denial-of-service attacks.

V05 hosted external form

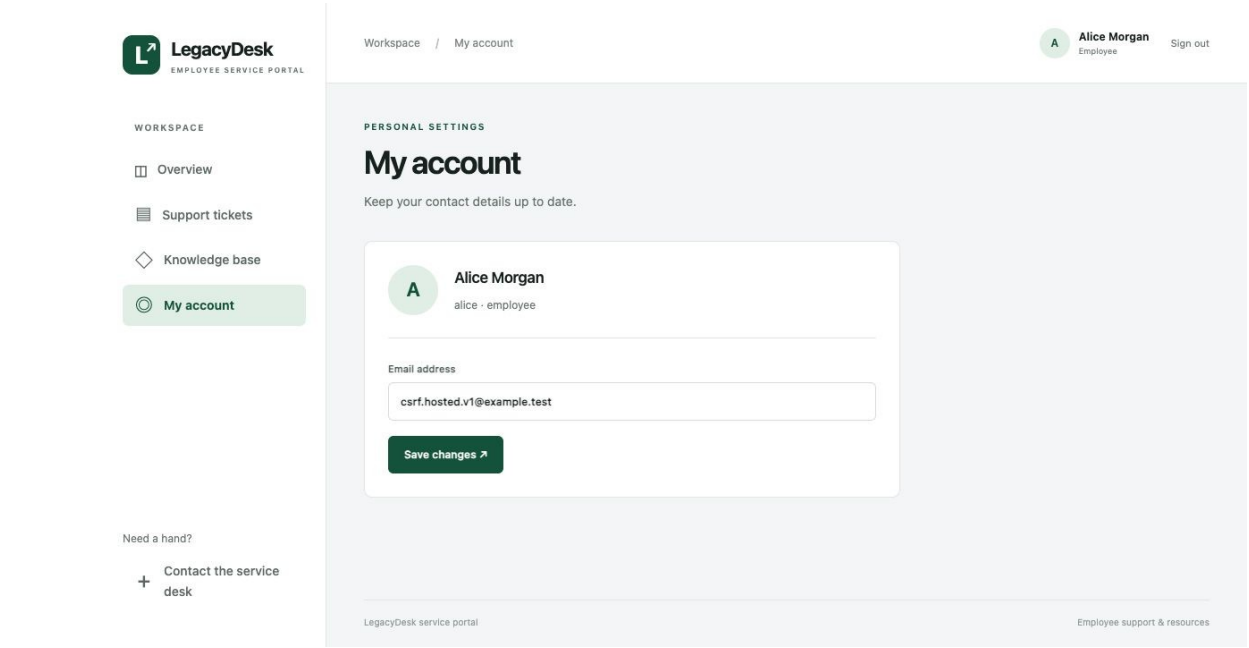
The CSRF demonstration uses a separate Render-hosted website. Each button submits one account-update request to the selected portal without a CSRF token. The page does not collect portal passwords or session cookies.



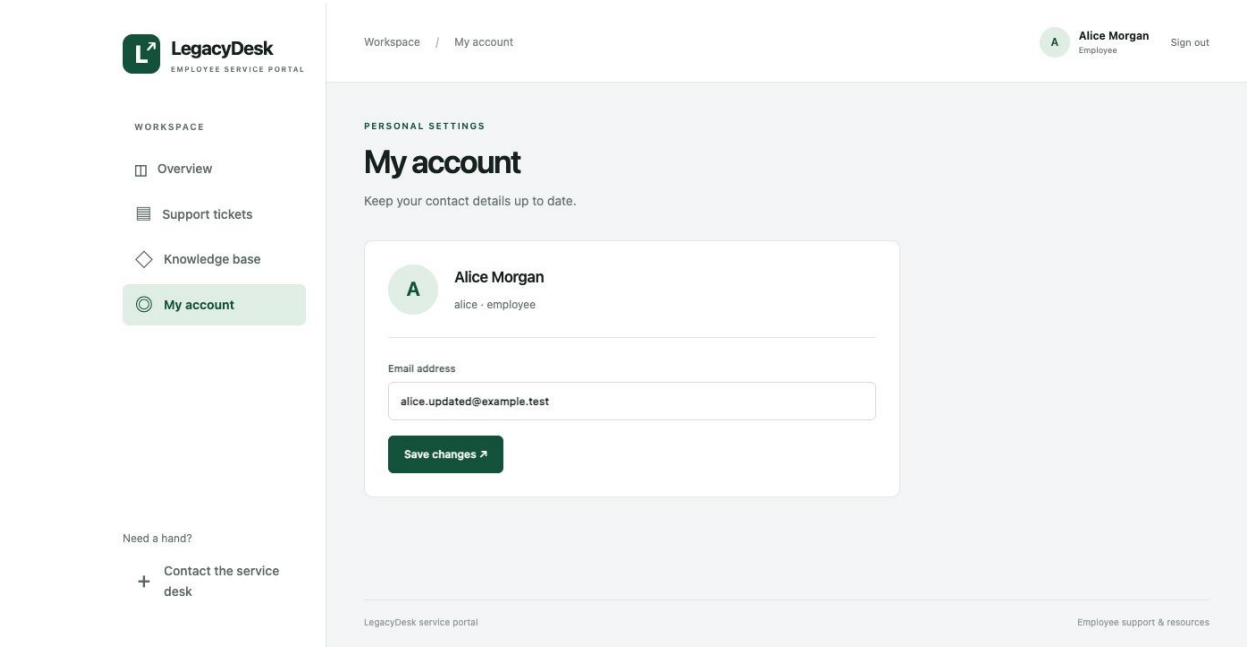
The hosted external form used for the V1 and V2 account-update comparison on 18 October 2026.

V05 hosted account state comparison

After submitting the external form, we refreshed the original authenticated account tabs. V1 showed the submitted email; V2 retained its previous email.



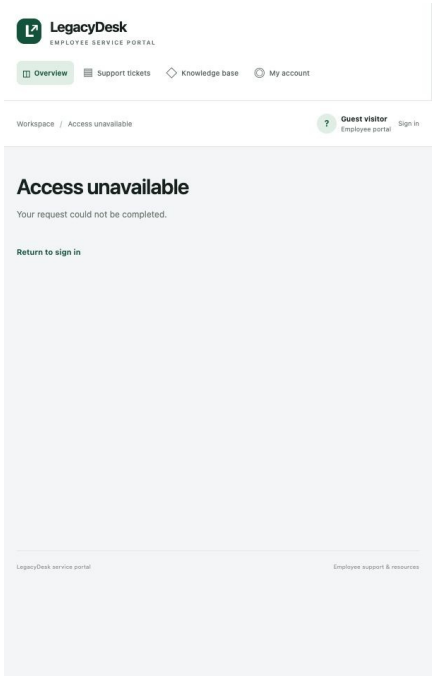
V1 accepted the external update and saved csrf.hosted.v1@example.test.



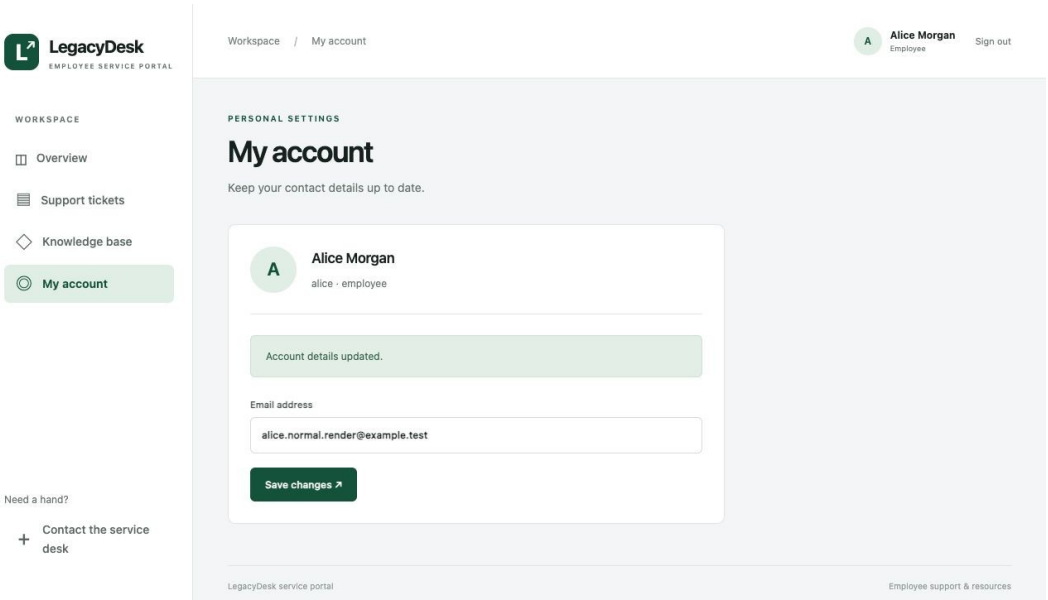
V2 retained alice.updated@example.test after the external request was rejected.

V05 rejected request and legitimate update

The external V2 submission was rejected. A later update made through the portal's own account form succeeded. These captures show the combined browser-cookie and server-side CSRF protections.



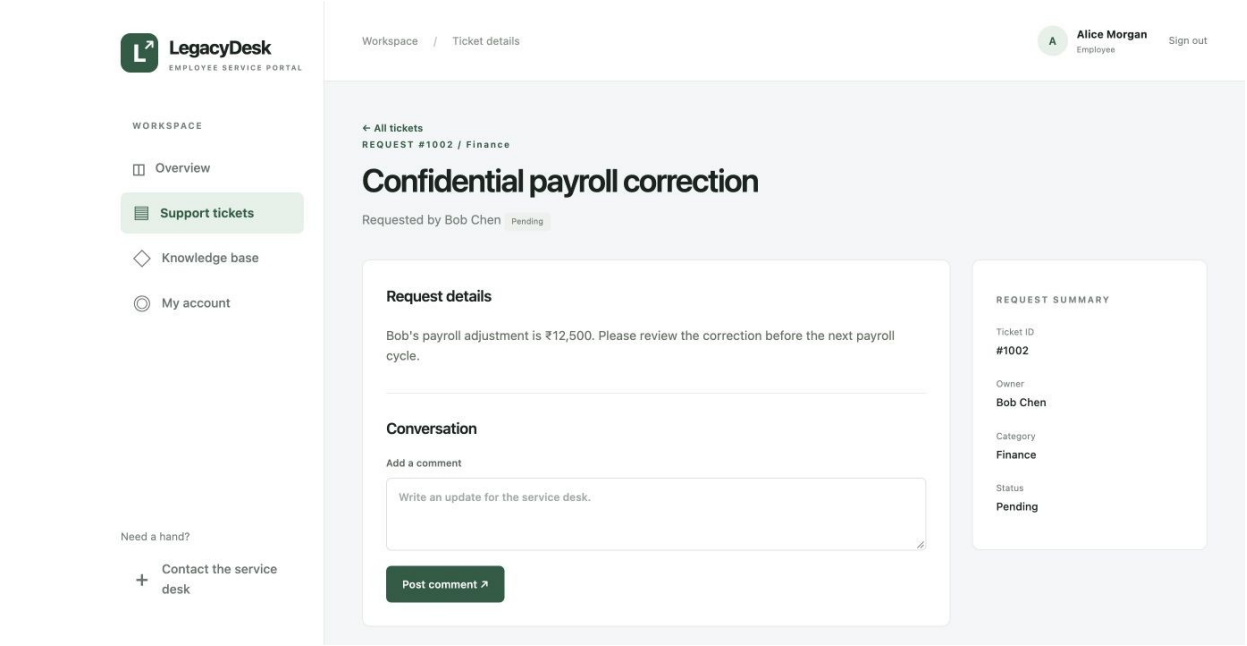
V2 response to the external form: Access unavailable.



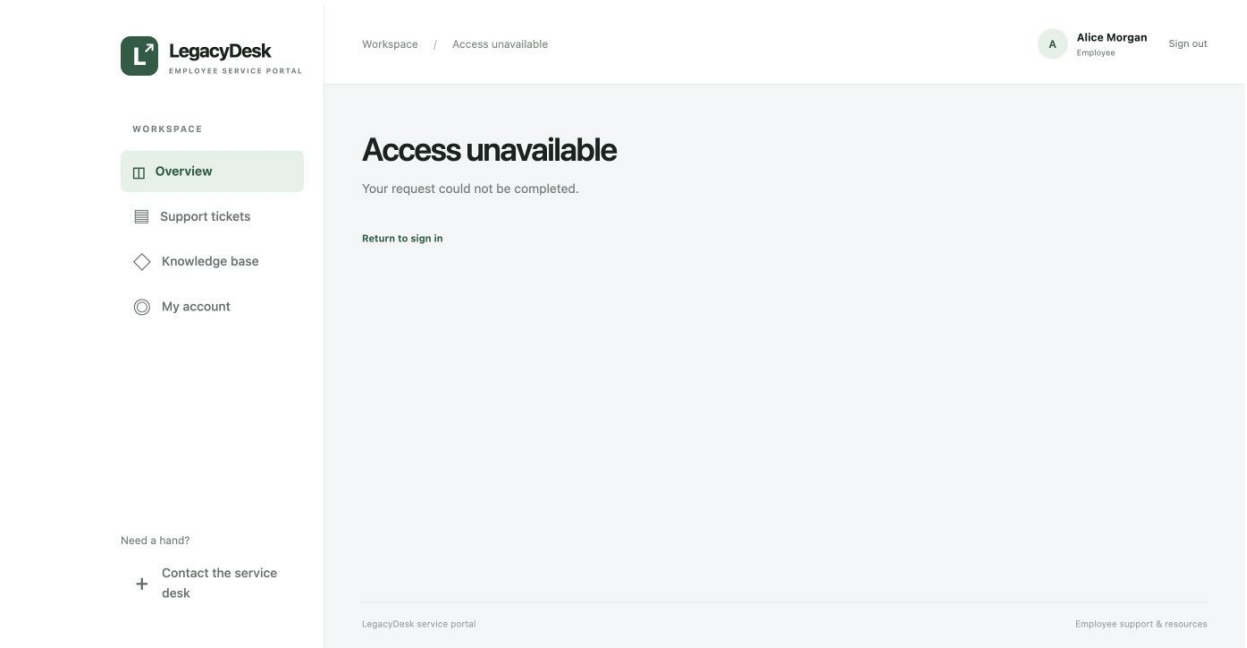
Captured confirmation area of a legitimate V2 account update: Account details updated.

V03 hosted ticket ownership comparison

These Render captures from 17 October 2026 compare the same ticket request using Alice's account. All employee and payroll records shown are fictional lab data.



V1 exposed Bob's ticket 1002 while the signed-in identity remained Alice.



V2 denied the same cross-account ticket request while Alice remained signed in.